

Customer Churn Prediction – Solution Approach Documentation

1. Problem Statement

The objective is to build a machine learning system capable of identifying customers likely to churn.

Customer churn prediction is a binary classification problem where:

- Positive Class (1) = Customer Churn
- Negative Class (-1) = Customer Retained

The primary business objective is to identify customers at risk of leaving so that retention actions can be taken proactively.

From a business perspective, missing a churner (False Negative) is generally more costly than incorrectly flagging a retained customer (False Positive), since acquiring new customers is typically more expensive than retaining existing ones.

2. Exploratory Data Analysis (EDA)

Dataset Overview

Property	Value
Rows	50,000
Features	231
Numerical Features	191
Categorical Features	38
Integer Features	1

Target Distribution

Class	Count
Retained (-1)	46,328
Churn (1)	3,672

Class Distribution:

- Retained: 92.66%

- Churn: 7.34%

Observation

The dataset is highly imbalanced.

A naive model predicting every customer as retained would achieve approximately 92.6% accuracy while failing to identify churners.

Therefore, accuracy is not a reliable evaluation metric for this problem. Precision, Recall, F1-Score and PR-AUC are more suitable.

Missing Value Analysis

Missing value analysis revealed extreme sparsity.

Summary statistics:

Metric	Missing %
Median Missingness	~97%
Maximum Missingness	100%
Minimum Missingness	0%

Several columns contained:

- 100% missing values
- 99% missing values
- 95% missing values

Examples:

- Var20
- Var39
- Var32
- Var31
- Var8
- Var15

All contained 100% missing data.

Observation

The dataset quality was the primary challenge.

Many features contained insufficient observations to reliably learn customer behavior.

Cardinality Analysis

Unique value analysis was performed to identify:

- Constant features
- High-cardinality features
- Potential identifier columns

Examples:

Feature	Unique Values
Var113	48,511
Var81	43,042
Var133	37,603
Var153	36,397

Observation

Features with extremely high cardinality may represent:

- Account IDs
- Transaction IDs

Such features risk introducing leakage and memorization rather than generalizable patterns.

Missingness Correlation Analysis

Missing value patterns were analyzed against churn labels.

Several variables showed different missing rates between retained and churned customers.

Example:

Feature	Missing (Retained)	Missing (Churn)
Var225	~51%	~61%

Observation

Missingness itself may carry predictive signal.

Instead of dropping all partially missing features, missing categories were preserved and treated as meaningful information.

3. Data Preprocessing

Step 1: Remove Non-Informative Features

Removed:

A. Fully Missing Features

Columns with:

- 100% missing values

Reason:

These features provide no information.

B. Constant Features

Columns containing only a single unique value.

Reason:

Constant features contain no variance and therefore no predictive power.

Step 2: Identify Feature Types

Separated:

Numerical Features

- float64
- int64

Categorical Features

- object

Final count after cleaning:

- 34 categorical features
- 173 numerical features

Total:

- 207 features
-

Step 3: Missing Value Handling

Numerical Features

CatBoost was allowed to handle numerical missing values natively.

Categorical Features

Missing values were converted to:

"Missing"

Reason:

CatBoost requires categorical values to be strings or integers.

Additionally, treating missing values as their own category allows the model to learn whether missingness itself is predictive.

4. Feature Engineering

Due to the lack of business metadata, feature engineering was intentionally conservative.

No synthetic business features were created because:

1. Variable meanings were unknown.
 2. Risk of creating non-interpretable features was high.
 3. Dataset sparsity already introduced significant uncertainty.
-

5. Validation Strategy

Stratified K-Fold Cross Validation

Configuration:

```
StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

Reason:

The dataset is highly imbalanced.

Stratification ensures every fold preserves the original churn ratio, producing more stable evaluation results.

Example:

Training:

- Retained: ~92.65%
- Churn: ~7.35%

Validation:

- Retained: ~92.66%
 - Churn: ~7.34%
-

6. Modelling Approach

Algorithm Selection

Chosen Model:

CatBoostClassifier

Reasons:

1. Native support for categorical variables.
 2. Native handling of numerical missing values.
 3. Strong performance on tabular datasets.
 4. Reduced preprocessing requirements.
 5. Robust gradient boosting architecture.
-

Initial Hyperparameters

```
CatBoostClassifier(  
    iterations=500,  
    learning_rate=0.05,  
    depth=6,  
    loss_function="Logloss",  
    eval_metric="F1",  
    random_seed=42  
)
```

Hyperparameter Justification

Depth = 6

Selected as a balanced starting point.

- Lower depth → underfitting
- Higher depth → increased overfitting risk

Learning Rate = 0.05

Conservative learning rate to encourage gradual learning.

F1 Evaluation Metric

Chosen because:

- Precision alone is insufficient.
- Recall alone is insufficient.
- Business costs were not explicitly defined.

F1 provides a balanced measure of both precision and recall.

7. Handling Class Imbalance

Approach 1: Class Weights

Calculated ratio:

$46328 / 3672 \approx 12.6$

Used:

`class_weights=[1, 12.6]`

Purpose:

Increase penalty for missing churners.

Observed Results

Confusion Matrix:

```
[[6575 2690]
 [ 287  448]]
```

Metrics:

Metric	Value
Precision	0.14
Recall	0.61
F1	0.23

Observation:

- Recall improved substantially.
- Precision dropped significantly.

Model became overly aggressive in predicting churn.

Approach 2: No Class Weights

Confusion Matrix:

```
[[9262   3]
 [ 728   7]]
```

Metrics:

Metric	Value
Precision	0.70
Recall	0.01
F1	0.02

Observation:

Model almost ignored churners.

This demonstrated the necessity of imbalance handling.

Approach 3: Threshold Optimization

Instead of relying on the default 0.5 threshold, multiple decision thresholds were evaluated.

Threshold	Precision	Recall	F1
0.3	0.091	0.937	0.165
0.4	0.110	0.805	0.194
0.5	0.143	0.610	0.231
0.6	0.193	0.386	0.258

Threshold	Precision	Recall	F1
0.7	0.277	0.192	0.227
0.8	0.410	0.044	0.079

Best observed F1:

Threshold = 0.6

F1 = 0.258

This confirmed that threshold tuning can significantly influence precision-recall tradeoffs in imbalanced classification problems.

8. Success Criteria & Evaluation Metrics

Why Accuracy Was Not Used

With 92.6% retained customers:

A model predicting every customer as retained would achieve:

Accuracy $\approx$ 92.6%

while identifying zero churners.

Therefore accuracy is misleading.

Primary Metrics

Recall

Measures:

Out of all actual churners,
how many were correctly identified?

Business relevance:

Missing churners can directly impact revenue.

Precision

Measures:

Out of all predicted churners,
how many actually churned?

Business relevance:

Low precision creates unnecessary retention costs.

F1 Score

Measures:

Balance between Precision and Recall

Used as primary optimization metric due to uncertainty around relative business costs.

PR-AUC

Recommended for future evaluation.

Reason:

PR-AUC is highly informative in imbalanced classification settings because it evaluates performance across all thresholds and focuses on minority-class detection.

9. Key Findings

1. Data quality was the largest challenge.
 2. Many features contained >95% missing values.
 3. Class imbalance strongly influenced model behavior.
 4. CatBoost effectively handled mixed data types.
 5. Class weighting significantly improved recall.
 6. Threshold tuning improved F1 performance.
 7. Feature quality likely limits model performance more than hyperparameter selection.
-

10. Future Improvements

1. Business-level feature understanding.
2. Feature importance analysis.

3. Hyperparameter optimization.
 4. PR-AUC driven model selection.
 5. Comparison with:
 - XGBoost
 - LightGBM
 - Random Forest
 6. Evaluate:
 - SMOTE
-

Conclusion

The exercise demonstrated a complete churn prediction workflow covering EDA, preprocessing, imbalance handling, validation strategy, model development, threshold optimization, and business-focused evaluation. The primary limitation was dataset quality rather than modelling capability, highlighting the importance of data understanding before extensive model tuning.